



actividad_jwt_cicd

28/04/2026

Pablo Ortiz Muñoz

Índice

Índice.....	1
Instrucciones de entrega y normas generales.....	3
Repositorio GitHub.....	3
Fase 1.....	3
JWT.....	3
¿Qué hace?.....	3
Estructura.....	3
Flujo de trabajo.....	4
Ventajas.....	4
Actualizar base de datos.....	4
Cambios.....	5
Relaciones Eloquent entre ambos modelos.....	5
Usar json.....	5
Models.....	5
Pelicula.....	6
Director.....	7
Controllers.....	7
DirectorController.....	8
PeliculaController.....	9
Migrations.....	9
Director.....	10
Pelicula.....	11
routes.....	11
Implementación JWT.....	12
Instalar paquete.....	12
Publicar la configuración.....	13
Generar la clave secreta.....	14
Cambios en user.php.....	14
Configuramos auth.php para que use jwt.....	14
Creamos authController.....	15
Cambios en las rutas.....	16
Cambios sugeridos por el profesor.....	18
Comprobación token thunder client.....	18



Demostrar que el acceso sin token es rechazado.....	20
Demostrar que tras logout el token queda invalidado.....	20
Refresco token.....	21
Tests de integración.....	21
tests/Feature/Auth/.....	21
que he cambiado para que funcionen los test.....	23
tests/Feature/Directores/.....	23
tests/Feature/Peliculas/.....	23
tests/Feature/Security/.....	24
Fase 2.....	25
CI/CD.....	25
Dev Containers.....	28

Instrucciones de entrega y normas generales

Repositorio GitHub

- Un único repositorio por pareja, con ambos miembros como colaboradores.
- Estructura de ramas recomendada: main (producción), develop (integración), feature/* (funcionalidades).
- Cada fase debe identificarse mediante un tag de Git (p. ej. v1.0-jwt y v2.0-cicd) o una rama específica indicada por el profesor.
- El fichero .env NO debe estar en el repositorio. Incluir siempre .env.example con las variables necesarias (sin valores reales).

Fase 1

JWT

¿Qué hace?

Permite enviar información entre dos partes de forma segura y compacta como un objeto JSON

Estructura

- **Header:** Indica qué algoritmo se usa para la firma.
- **Payload:** Contiene los datos del usuario (como su ID o nombre) y la fecha de expiración.
- **Signature:** Es la firma digital que garantiza que el token no ha sido alterado.

Flujo de trabajo

Instalación: Instalaremos la librerías jwt y configuraremos el modelo User para que sea compatible.

Autenticación: Crearemos un endpoint de login que, si las credenciales son válidas, devolverá un access_token.

Protección: Aplicaremos el middleware auth:api a tus rutas de Películas y Directores.

Ventajas

Escalabilidad: Como el servidor no guarda sesiones, es mucho más fácil escalar la aplicación.

Seguridad: Evita ataques de sesión comunes y permite que clientes externos se comuniquen de forma estándar con tu API.

Actualizar base de datos

Cada vez que se hacen cambios con la base de datos hay que actualizarla con el siguiente comando

```

VM 2026 - CIBER-portmon413-Ubuntu22 (1)
4 de may 03:37
Visual Studio Code
Edit Selection View Go Run Terminal Help
pelicapp-main
EXPLORER
  pelicapp-main
    app
    Models
    Pelicula.php
    User.php
    Providers
    View/Components
    AppLayout.php
    GuestLayout.php
    Table.php
    bootstrap
    config
    database
    factories
    migrations
    seeders
    DatabaseSeeder.php
    DirectorSeeder.php
    FilmSeeder.php
    gitignore
    database.sqlite
    node_modules
    public
    resources
    routes
    auth.php
    console.php
    web.php
    storage
    tests
    vendor
    editorconfig
    env
    env.example
    gitattributes
    gitignore
    artisan
    OUTLINE
    TIMELINE
routes.php
1 <?php
2
3 use App\Http\Controllers\DirectorController;
4 use App\Http\Controllers\PeliculaController;
5 use App\Http\Controllers\ProfileController;
6 use Illuminate\Support\Facades\Route;
7
8
9
10
11 Route::get('/', function () {
12     return view('welcome');
13 });
14
15 // Rutas de Dashboard
16 Route::get('/dashboard', function () {
17     return view('dashboard');
18 })->middleware(['auth', 'verified'])->name('dashboard');
19
20 // Rutas de Perfil
21 Route::middleware('auth')->group(function () {
22     Route::get('/profile', [ProfileController::class, 'edit'])->name('profile.edit');
23     Route::patch('/profile', [ProfileController::class, 'update'])->name('profile.update');
24     Route::delete('/profile', [ProfileController::class, 'destroy'])->name('profile.destroy');
25 });
26
27 /**
28  *
29  *resource para habilitar index, store, show, update y destroy.
30
terminal
usuario@VM-Ubuntu:~/Descargas/pelicapp-main$ php artisan migrate:--fresh
[INFO] Preparing database.
Creating migration table ..... 19.87ms DONE
[INFO] Running migrations.
0001_01_01_000000 create users table ..... 72.58ms DONE
0001_01_01_000001 create cache table ..... 58.19ms DONE
0001_01_01_000002 create jobs table ..... 183.96ms DONE
2026_03_23_153317 create directors table ..... 78.60ms DONE
2026_03_23_153328 create films table ..... 65.58ms DONE

```

php artisan migrate:--fresh

Cambios

Relaciones Eloquent entre ambos modelos

Cambios: He definido métodos dentro de las clases de los modelos para conectar las tablas. En el modelo Director añadí la función películas() y en el modelo Pelicula la función director().

Cómo funciona: Laravel usa estas funciones para saber que un director tiene muchas películas (hasMany) y que una película pertenece a un único director (belongsTo).

Para qué sirve: Sirve para que, al pedir un director, el código pueda traer automáticamente todas sus películas.

Usar json

Qué he cambiado: He modificado los controladores para que, en lugar de devolver una vista, devuelvan una respuesta usando response()->json().

Cómo funciona: El servidor da los datos en formato JSON.

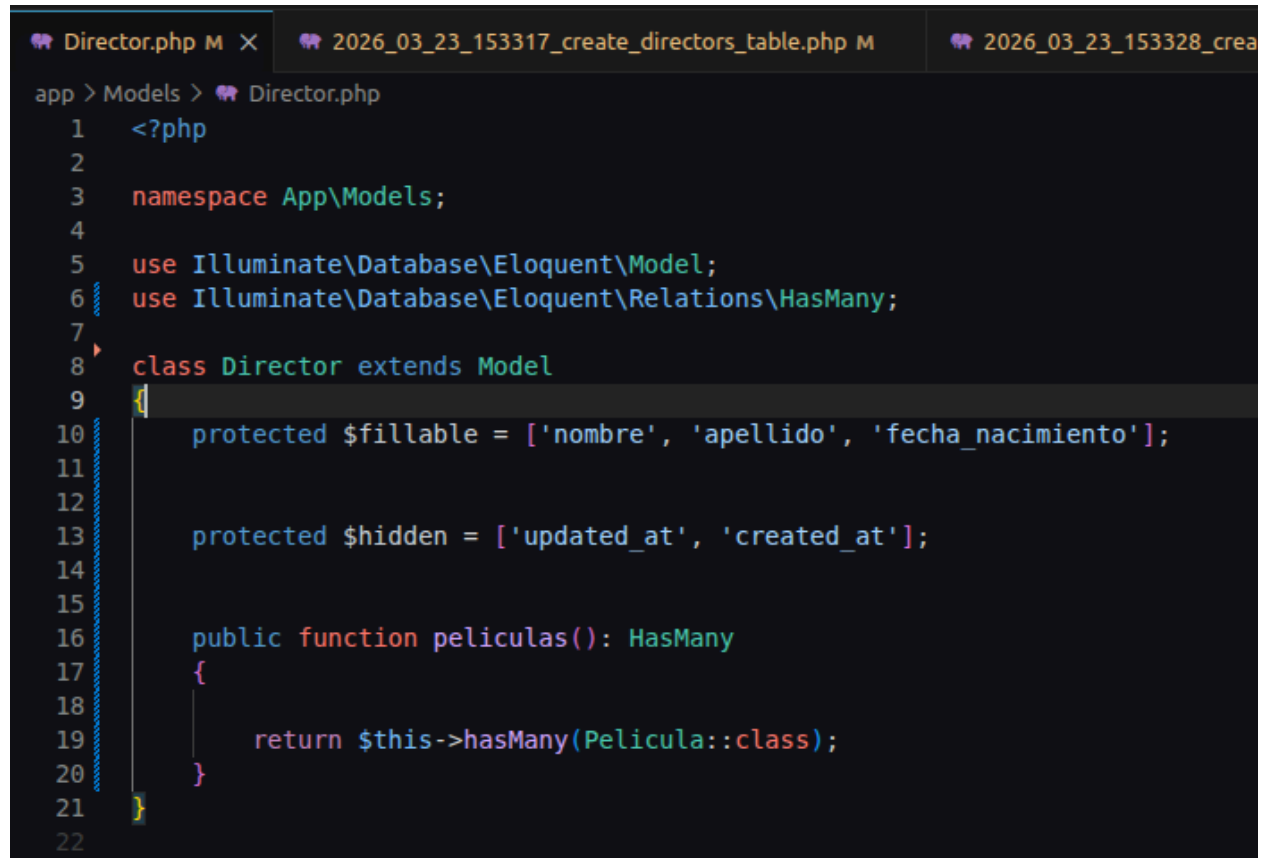
Para qué sirve: Para que mi proyecto funcione como una API. Esto permite que los datos puedan ser consumidos por cualquier otro programa o interfaz sin problemas.

Models

Pelicula

```
Director.php M × Pelicula.php U × 2026_03_23_153317_create_direct
app > Models > Pelicula.php
1  <?php
2
3  namespace App\Models;
4
5  use Illuminate\Database\Eloquent\Model;
6  use Illuminate\Database\Eloquent\Relations\BelongsTo;
7
8  class Pelicula extends Model
9  {
10
11      protected $table = 'peliculas';
12
13
14      protected $fillable = [
15          'title',
16          'release_date',
17          'sinopsis',
18          'duration',
19          'genre',
20          'director_id'
21      ];
22
23      protected $hidden = ['updated_at', 'created_at'];
24
25      public function director(): BelongsTo
26      {
27          return $this->belongsTo(Director::class);
28      }
29  }
```

Director



```
app > Models > Director.php
1  <?php
2
3  namespace App\Models;
4
5  use Illuminate\Database\Eloquent\Model;
6  use Illuminate\Database\Eloquent\Relations\HasMany;
7
8  class Director extends Model
9  {
10     protected $fillable = ['nombre', 'apellido', 'fecha_nacimiento'];
11
12
13     protected $hidden = ['updated_at', 'created_at'];
14
15
16     public function peliculas(): HasMany
17     {
18         return $this->hasMany(Pelicula::class);
19     }
20
21 }
22
```

Controllers

Cambios: 5 métodos estándar: index, store, show, update y destroy

DirectorController

```

app > Http > Controllers > DirectorController.php
1  <?php
2
3  namespace App\Http\Controllers;
4
5  use Illuminate\Http\Request;
6  use App\Models\Director;
7  use Illuminate\Http\JsonResponse;
8
9  class DirectorController extends Controller
10 {
11     public function index(): JsonResponse
12     {
13         $directores = Director::all();
14         return response()->json($directores, 200);
15     }
16
17     public function store(Request $request): JsonResponse
18     {
19         //Validación y almacenamiento
20         $validated = $request->validate([
21             'nombre' => 'required|string|max:255',
22             'apellido' => 'required|string|max:255',
23             'fecha_nacimiento' => 'required|date',
24         ]);
25
26         $director = Director::create($validated);
27         return response()->json($director, 201);
28     }
29
30     public function show(string $id): JsonResponse
31     {
32         //Carga la relación películas definida en el modelo
33         $director = Director::with('películas')->find($id);
34
35         if (!$director) {
36             return response()->json(['message' => 'Director no encontrado'], 404);
37         }
38
39         return response()->json($director, 200);
40     }
41
42     public function update(Request $request, string $id): JsonResponse
43     {
44         $director = Director::find($id);
45
46         if (!$director) {
47             return response()->json(['message' => 'Director no encontrado'], 404);
48         }
49
50         $director->update($request->all());
51         return response()->json($director, 200);
52     }
53
54     public function destroy(string $id): JsonResponse
55     {
56         $director = Director::find($id);
57
58         if (!$director) {
59             return response()->json(['message' => 'Director no encontrado'], 404);
60         }
61
62         $director->delete();
63         return response()->json(['message' => 'Director eliminado correctamente'], 200);
64     }
65 }

```

PeliculaController

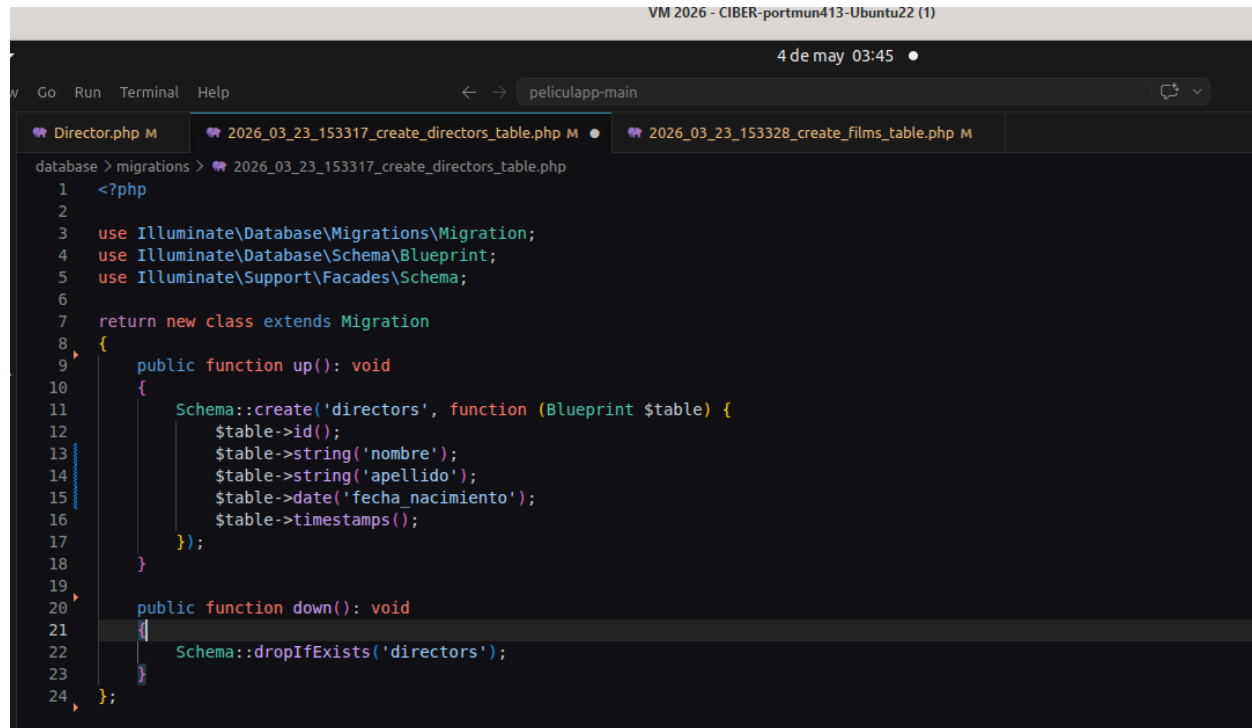
```

1  <?php
2
3  namespace App\Http\Controllers;
4
5  use App\Models\Pelicula;
6  use Illuminate\Http\Request;
7  use Illuminate\Http\JsonResponse;
8
9  class PeliculaController extends Controller
10 {
11     /**
12      * Listado de peliculas con su director
13      */
14     public function index(): JsonResponse
15     {
16         return response()->json(Pelicula::with('director')->get(), 200);
17     }
18
19     /**
20      * Crear nueva pelicula
21      */
22     public function store(Request $request): JsonResponse
23     {
24         $validated = $request->validate([
25             'title' => 'required|string|max:255',
26             'release_date' => 'required|date',
27             'sinopsis' => 'required|string',
28             'duration' => 'required|integer',
29             'genre' => 'required|string',
30             'director_id' => 'required|exists:directors,id',
31         ]);
32
33         $pelicula = Pelicula::create($validated);
34         return response()->json($pelicula, 201);
35     }
36
37     /**
38      * Mostrar una pelicula especifica
39      */
40     public function show($id): JsonResponse
41     {
42         $pelicula = Pelicula::with('director')->find($id);
43
44         if (!$pelicula) {
45             return response()->json(['error' => 'Pelicula no encontrada'], 404);
46         }
47
48         return response()->json($pelicula, 200);
49     }
50
51     /**
52      * Actualizar pelicula
53      */
54     public function update(Request $request, $id): JsonResponse
55     {
56         $pelicula = Pelicula::find($id);
57
58         if (!$pelicula) {
59             return response()->json(['error' => 'Pelicula no encontrada'], 404);
60         }
61
62         $pelicula->update($request->all());
63         return response()->json($pelicula, 200);
64     }
65
66     /**
67      * Eliminar pelicula
68      */
69     public function destroy($id): JsonResponse
70     {
71         $pelicula = Pelicula::find($id);
72
73         if (!$pelicula) {
74             return response()->json(['error' => 'Pelicula no encontrada'], 404);
75         }
76
77         $pelicula->delete();
78         return response()->json(['message' => 'Eliminada correctamente'], 200);
79     }
80 }

```

Migrations

Director



The screenshot shows a code editor window titled "VM 2026 - CIBER-portmun413-Ubuntu22 (1)" with a timestamp of "4 de may 03:45". The editor has tabs for "Director.php M", "2026_03_23_153317_create_directors_table.php M", and "2026_03_23_153328_create_films_table.php M". The active tab is "2026_03_23_153317_create_directors_table.php M". The code is a PHP migration file for creating a directors table. It includes the following code:

```
database > migrations > 2026_03_23_153317_create_directors_table.php
1  <?php
2
3  use Illuminate\Database\Migrations\Migration;
4  use Illuminate\Database\Schema\Blueprint;
5  use Illuminate\Support\Facades\Schema;
6
7  return new class extends Migration
8  {
9      public function up(): void
10     {
11         Schema::create('directors', function (Blueprint $table) {
12             $table->id();
13             $table->string('nombre');
14             $table->string('apellido');
15             $table->date('fecha_nacimiento');
16             $table->timestamps();
17         });
18     }
19
20     public function down(): void
21     {
22         Schema::dropIfExists('directors');
23     }
24 };
```

Pelicula

```

database > migrations > 2026_03_23_153328_create_films_table.php
1  <?php
2
3  use Illuminate\Database\Migrations\Migration;
4  use Illuminate\Database\Schema\Blueprint;
5  use Illuminate\Support\Facades\Schema;
6
7  return new class extends Migration
8  {
9
10     public function up(): void
11     {
12         Schema::create('peliculas', function (Blueprint $table) {
13             $table->id();
14             $table->string('title');
15             $table->date('release_date');
16             $table->text('sinopsis');
17             $table->integer('duration');
18             $table->string('gendre');
19             // Relación con la tabla directors
20             $table->foreignId('director_id')
21                 ->constrained('directors')
22                 ->onUpdate('cascade')
23                 ->onDelete('restrict');
24             $table->timestamps();
25         });
26     }
27
28     public function down(): void
29     {
30
31         Schema::dropIfExists('peliculas');
32     }
33
34 };

```

routes

Qué he cambiado: En lugar de escribir cada ruta una por una, he utilizado `Route::resource` para los directores y las películas.

Cómo funciona: Esta sola línea crea automáticamente todas las URLs necesarias como `/directores` para ver el listado o `/directores/{id}` para borrar y las conecta con su método correspondiente en el controlador.

Para qué sirve: Para ahorrar código y evitar errores. Es la forma más limpia y rápida de exponer todas las funciones del CRUD hacia el exterior.

Implementación JWT

Instalar paquete

```

VM 2026 - CIBER-portmun413-Ubuntu22 (1)
4 de may 03:56
Go Run Terminal Help
← → peliculapp-main
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
• usuario@MV-Ubuntu:~/Descargas/peliculapp-main$ composer require php-open-source-saver/jwt-auth
./composer.json has been updated
Running composer update php-open-source-saver/jwt-auth
Loading composer repositories with package information
Updating dependencies
Lock file operations: 4 installs, 0 updates, 0 removals
  - Locking lcobucci/jwt (5.6.0)
  - Locking namshi/jose (7.2.3)
  - Locking php-open-source-saver/jwt-auth (v2.9.0)
  - Locking symfony/polyfill-php56 (v1.20.0)
Writing lock file
Installing dependencies from lock file (including require-dev)
Package operations: 4 installs, 0 updates, 0 removals
  - Downloading namshi/jose (7.2.3)
  - Downloading lcobucci/jwt (5.6.0)
  - Downloading php-open-source-saver/jwt-auth (v2.9.0)
  - Installing symfony/polyfill-php56 (v1.20.0)
  - Installing namshi/jose (7.2.3): Extracting archive
  - Installing lcobucci/jwt (5.6.0): Extracting archive
  - Installing php-open-source-saver/jwt-auth (v2.9.0): Extracting archive
2 package suggestions were added by new dependencies, use `composer suggest` to see details.
Generating optimized autoload files
> Illuminate\Foundation\ComposerScripts::postAutoloadDump
> @php artisan package:discover --ansi

  INFO  Discovering packages.

laravel/breeze .....
laravel/pail .....
laravel/tinker .....
nesbot/carbon .....
nunomaduro/collision .....
nunomaduro/termwind .....
pestphp/pest-plugin-laravel .....
php-open-source-saver/jwt-auth .....

86 packages you are using are looking for funding.
Use the `composer fund` command to find out more!
> @php artisan vendor:publish --tag=laravel-assets --ansi --force

  INFO  No publishable resources for tag [laravel-assets].

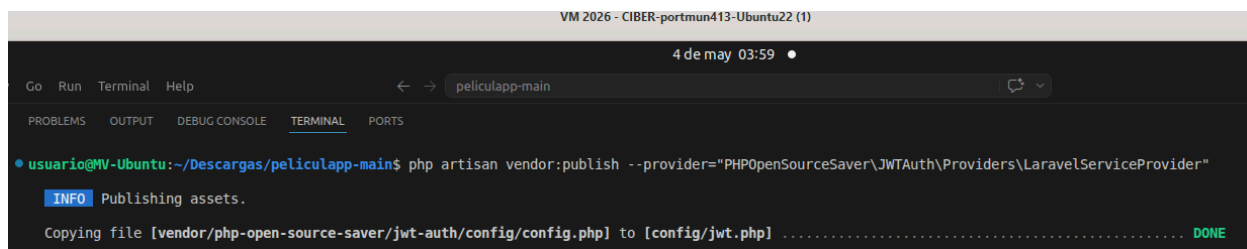
No security vulnerability advisories found.
Using version ^2.9 for php-open-source-saver/jwt-auth
o usuario@MV-Ubuntu:~/Descargas/peliculapp-main$

```

Publicar la configuración

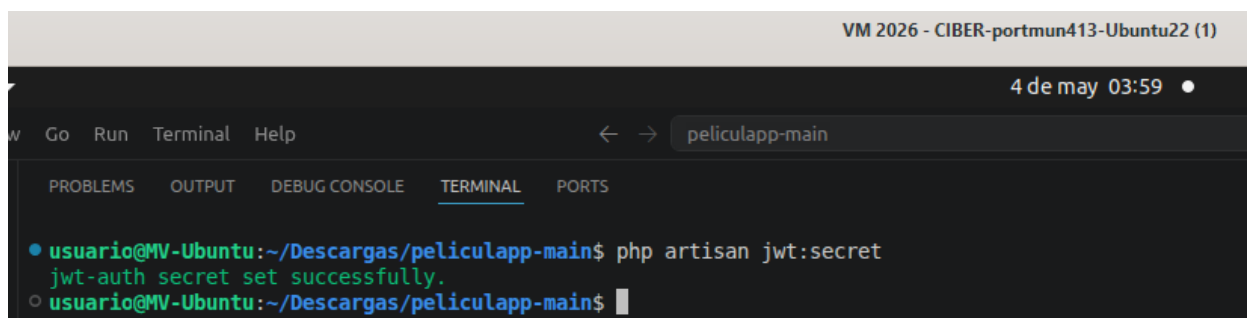
php artisan vendor:publish

--provider="PHPOpenSourceSaver\JWTAuth\Providers\LaravelServiceProvider"



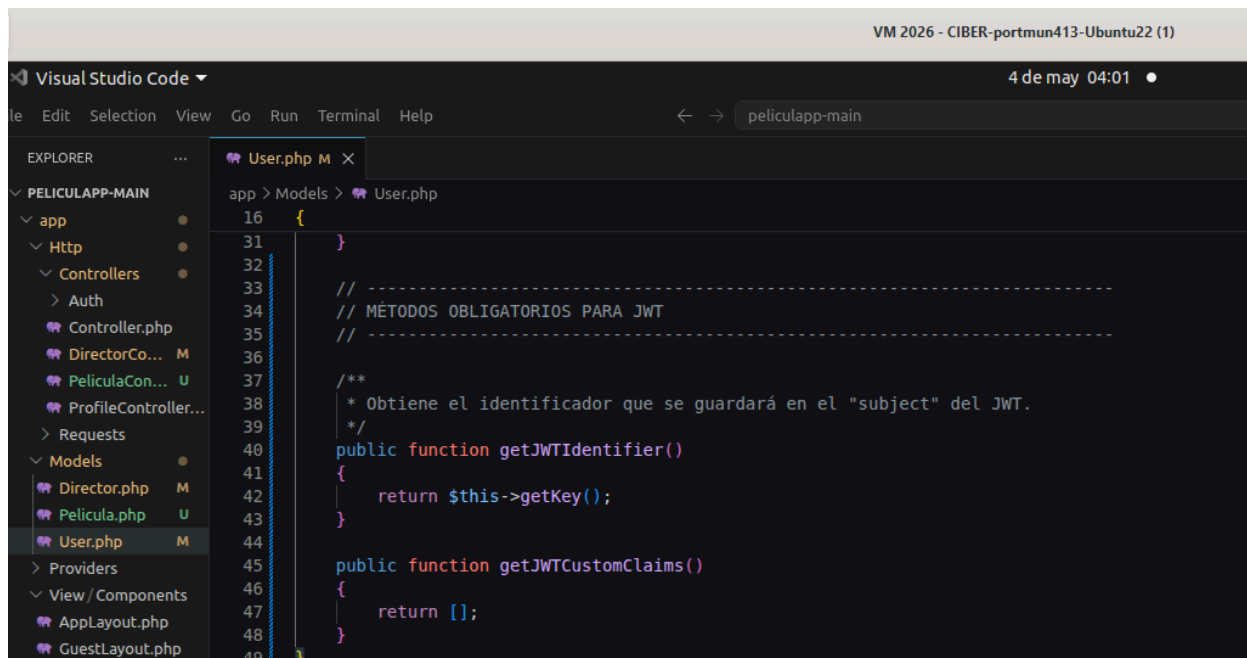
```
VM 2026 - CIBER-portmun413-Ubuntu22 (1)
4 de may 03:59
Go Run Terminal Help
← → peliculapp-main
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
• usuario@MV-Ubuntu:~/Descargas/peliculapp-main$ php artisan vendor:publish --provider="PHPOpenSourceSaver\JWTAuth\Providers\LaravelServiceProvider"
INFO Publishing assets.
Copying file [vendor/php-open-source-saver/jwt-auth/config/config.php] to [config/jwt.php] ..... DONE
```

Generar la clave secreta



```
VM 2026 - CIBER-portmun413-Ubuntu22 (1)
4 de may 03:59
Go Run Terminal Help
← → peliculapp-main
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
• usuario@MV-Ubuntu:~/Descargas/peliculapp-main$ php artisan jwt:secret
jwt-auth secret set successfully.
o usuario@MV-Ubuntu:~/Descargas/peliculapp-main$
```

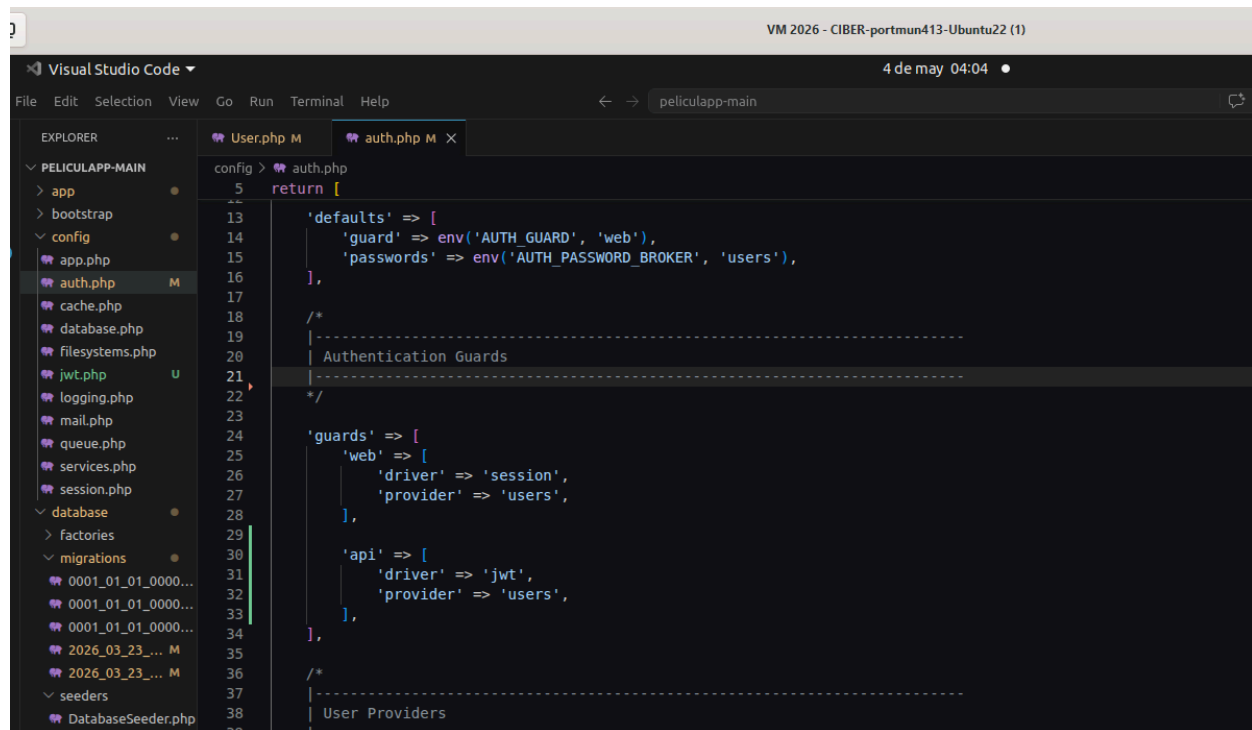
Cambios en user.php



```
VM 2026 - CIBER-portmun413-Ubuntu22 (1)
4 de may 04:01
Visual Studio Code
File Edit Selection View Go Run Terminal Help
← → peliculapp-main
EXPLORER
PELICULAPP-MAIN
app
  Http
  Controllers
    Auth
      Controller.php
      DirectorCo... M
      PeliculaCon... U
      ProfileController...
    Requests
  Models
    Director.php M
    Pelicula.php U
    User.php M
  Providers
  View/Components
    AppLayout.php
    GuestLayout.php
app > Models > User.php
16 {
31 }
32
33 // -----
34 // MÉTODOS OBLIGATORIOS PARA JWT
35 // -----
36
37 /**
38  * Obtiene el identificador que se guardará en el "subject" del JWT.
39  */
40 public function getJWTIdentifier()
41 {
42     return $this->getKey();
43 }
44
45 public function getJWTCustomClaims()
46 {
47     return [];
48 }
49 }
```

ponemos los métodos obligatorios de jwt

Configuramos auth.php para que use jwt



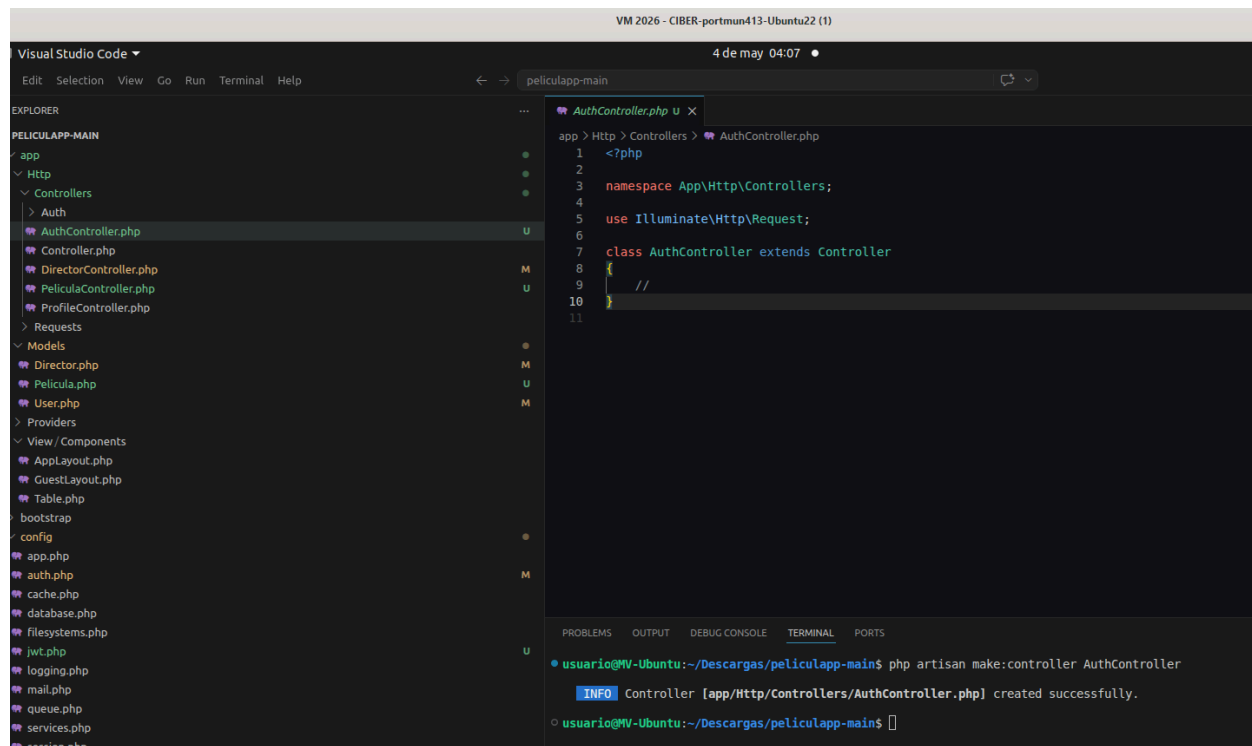
Visual Studio Code interface showing the file explorer on the left with the project structure of 'PELICULAPP-MAIN'. The file 'auth.php' is selected in the Explorer. The main editor displays the code for 'auth.php'.

```

5 return [
13     'defaults' => [
14         'guard' => env('AUTH_GUARD', 'web'),
15         'passwords' => env('AUTH_PASSWORD_BROKER', 'users'),
16     ],
17
18     /*
19     |-----
20     | Authentication Guards
21     |-----
22     */
23
24     'guards' => [
25         'web' => [
26             'driver' => 'session',
27             'provider' => 'users',
28         ],
29
30         'api' => [
31             'driver' => 'jwt',
32             'provider' => 'users',
33         ],
34     ],
35
36     /*
37     |-----
38     | User Providers
39     |-----

```

Creamos authController



Visual Studio Code interface showing the file explorer on the left with the project structure of 'PELICULAPP-MAIN'. The file 'AuthController.php' is selected in the Explorer. The main editor displays the code for 'AuthController.php'.

```

1 <?php
2
3 namespace App\Http\Controllers;
4
5 use Illuminate\Http\Request;
6
7 class AuthController extends Controller
8 {
9     //
10 }
11

```

Terminal output:

```

usuario@MV-Ubuntu:~/Descargas/peliculapp-main$ php artisan make:controller AuthController
INFO Controller [app/Http/Controllers/AuthController.php] created successfully.
usuario@MV-Ubuntu:~/Descargas/peliculapp-main$

```



```

4 de may 04:08 •
liculapp-main

AuthController.php x
app > Http > Controllers > AuthController.php
1  <?php
2
3  namespace App\Http\Controllers\Api;
4
5  use App\Http\Controllers\Controller;
6  use Illuminate\Http\Request;
7  use Illuminate\Support\Facades\Auth;
8
9  class AuthController extends Controller
10 {
11     /**
12      * Iniciar sesión y obtener el token JWT
13      */
14     public function login(Request $request)
15     {
16         $credentials = $request->validate([
17             'email' => 'required|email',
18             'password' => 'required',
19         ]);
20
21         // Intentamos autenticar con el guard 'api' (JWT)
22         if (!$token = auth('api')->attempt($credentials)) {
23             return response()->json(['error' => 'Credenciales inválidas'], 401);
24         }
25
26         return response()->json([
27             'access_token' => $token,
28             'token_type' => 'bearer',
29             'expires_in' => auth('api')->factory()->getTTL() * 60
30         ], 200);
31     }
32
33     /**
34      * Cerrar sesión (invalidar el token)
35      */
36     public function logout()
37     {
38         auth('api')->logout();
39         return response()->json(['message' => 'Sesión cerrada correctamente']);
40     }
41 }

```

en este archivo va toda la lógica de jwt para la autenticación

Cambios en las rutas

```
// Ruta pública para obtener el token de acceso
Route::post('login', [AuthController::class, 'login']);

/**
 * Rutas protegidas por JWT.
 * Solo los usuarios con un token válido pueden acceder a estos recursos.
 */
Route::middleware('auth:api')->group(function () {

    // CRUD completo de Directores
    Route::resource('directores', DirectorController::class);

    // CRUD completo de Películas
    Route::resource('peliculas', PeliculaController::class);

    // Ruta para cerrar sesión en la API
    Route::post('logout', [AuthController::class, 'logout']);
});

require __DIR__ . '/auth.php';
```

Cambios realizados en las Rutas

Importación del AuthController: He añadido una línea para que el archivo de rutas sepa dónde está el controlador que gestiona los tokens JWT.

Creación de la ruta de Login: He añadido `Route::post('login', ...)` para el login y que funcione el token jwt

Implementación del Middleware `auth:api`: He metido las rutas de directores y películas dentro de un grupo protegido.

Cambio de Seguridad: Antes las rutas eran públicas; ahora, gracias al middleware, Laravel exige un token JWT válido para dejar pasar la petición.

¿Cómo funciona ahora?

He organizado las rutas en dos: una parte pública para el Login y una privado para los Recursos. Cuando alguien intenta acceder a /películas, el sistema ahora consulta al JWT en lugar de buscar una sesión. Si no se envía el token generado en el login, el sistema corta el acceso.

Cambios sugeridos por el profesor

- No poner todas las rutas en web.php e instalar api.php para trabajar con las de /api

The screenshot shows the Visual Studio Code editor with the file explorer on the left displaying the project structure. The main editor window shows the contents of `web.php` in the `routes` directory. The code defines routes for a public area (welcome) and a private area (dashboard, profile) protected by the 'auth' middleware. The terminal at the bottom shows the execution of `php artisan install:api`, which successfully installs the API routes and publishes the API routes file.

```

1 <?php
2
3 use App\Http\Controllers\ProfileController;
4 use Illuminate\Support\Facades\Route;
5
6
7 Route::get('/', function () {
8     return view('welcome');
9 });
10
11 // Rutas de Dashboard
12 Route::get('/dashboard', function () {
13     return view('dashboard');
14 }->middleware(['auth', 'verified'])->name('dashboard');
15
16 // Rutas de Perfil de Usuario
17 Route::middleware('auth')->group(function () {
18     Route::get('/profile', [ProfileController::class, 'edit'])->name('profile.edit');
19     Route::patch('/profile', [ProfileController::class, 'update'])->name('profile.update');
20     Route::delete('/profile', [ProfileController::class, 'destroy'])->name('profile.destroy');
21 });
22
23 require __DIR__.'/auth.php';
  
```

```

usuario@MV-Ubuntu:~/Descargas/peliculapp-main$ php artisan install:api
php-open-source-saver/jwt-auth ..... DONE

86 packages you are using are looking for funding.
Use the 'composer fund' command to find out more!
> @php artisan vendor:publish --tag=laravel-assets --ansi --force

INFO No publishable resources for tag [laravel-assets].

No security vulnerability advisories found.

INFO Published API routes file.

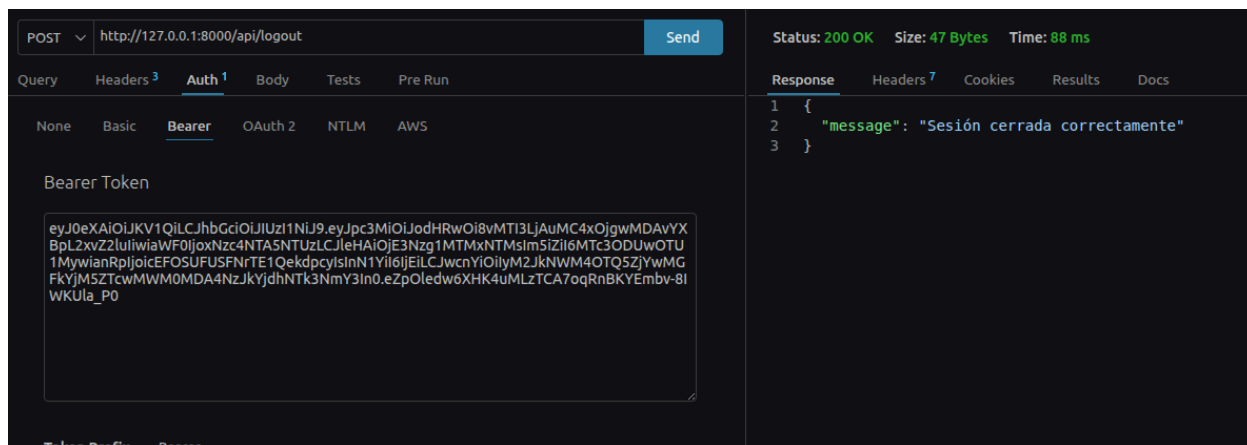
One new database migration has been published. Would you like to run all pending database migrations? (yes/no) [yes]:
  
```

Comprobación token thunder client

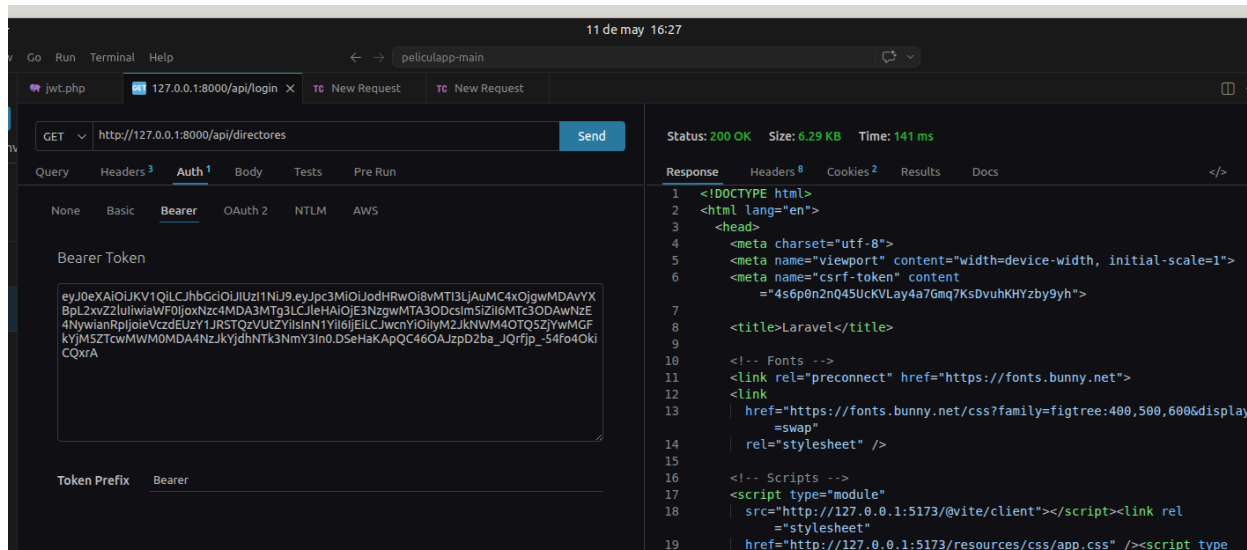


VM 2026 - CIBER-portmun413-Ubuntu22 (1)

Demstrar que tras logout el token queda invalidado



al hacer logout cuando intento entrar con el token no me deja entrar



Refresco token

Como el token expira en 60 min tendría que gestionar un token de refresco para que no haya que generarlo otra vez

Tests de integración

tests/Feature/Auth/

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
• usuario@MV-Ubuntu:~/Descargas/peliculapp-main$ php artisan make:test Auth\ApiAuthTest

  INFO  Test [tests/Feature/Auth/ApiAuthTest.php] created successfully.

• usuario@MV-Ubuntu:~/Descargas/peliculapp-main$ S
```

```
<?php

namespace Tests\Feature\Auth;

use App\Models\User;
use Illuminate\Foundation\Testing\RefreshDatabase;
use Tests\TestCase;

class ApiAuthTest extends TestCase
{
    use RefreshDatabase;

    protected function setUp(): void
    {
        parent::setUp();
        // Creamos un usuario base para todas las pruebas
        $this->user = User::factory()->create([
            'email' => 'test@ejemplo.com',
            'password' => bcrypt('admin123'),
        ]);
    }

    /** 1. Login válido devuelve token */
    public function test_login_con_credenciales_validas_devuelve_token()
    {
        $response = $this->postJson('/api/login', [
            'email' => 'test@ejemplo.com',
            'password' => 'admin123',
        ]);
    }
}
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
• usuario@MV-Ubuntu:~/Descargas/peliculapp-main$ php artisan test tests/Feature/Auth/ApiAuthTest.php

  PASS  Tests\Feature\Auth\ApiAuthTest
  ✓ login con credenciales validas devuelve token
  ✓ login con credenciales invalidas devuelve 401
  ✓ login con campos faltantes devuelve 422
  ✓ logout invalida el token
  ✓ refresh devuelve nuevo token valido
  ✓ me devuelve datos del usuario autenticado
  ✓ acceso sin token devuelve 401
  ✓ acceso con token malformado devuelve 401

  Tests: 8 passed (16 assertions)
  Duration: 0.19s

• usuario@MV-Ubuntu:~/Descargas/peliculapp-main$
```

que he cambiado para que funcionen los test

En el controlador de autenticación:

Método me(): Permite que un usuario autenticado recupere sus propios datos (nombre, email) enviando su token. Es fundamental para que el sistema sepa quién está operando.

Método refresh(): Permite renovar un token antes de que caduque. Esto es una medida de seguridad esencial en entornos de ciberseguridad para no obligar al usuario a hacer login constantemente, pero manteniendo los tokens con una vida corta.

Función auxiliar respondWithToken(): Centralizamos cómo se responde al cliente. Ahora, tanto el login como el refresh devuelven siempre la misma estructura JSON

tests/Feature/Directores/

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
• usuario@MV-Ubuntu:~/Descargas/peliculapp-main$ php artisan make:test Directores/DirectorApiTest

  INFO  Test [tests/Feature/Directores/DirectorApiTest.php] created successfully.

○ usuario@MV-Ubuntu:~/Descargas/peliculapp-main$ S

```

```

• usuario@MV-Ubuntu:~/Descargas/peliculapp-main$ php artisan make:factory PeliculaFactory

  INFO  Factory [database/factories/PeliculaFactory.php] created successfully.

```

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
usuario@MV-Ubuntu:~/Descargas/peliculapp-main$ php artisan test tests/Feature/Directores/DirectorApiTest.php
✓ crear director con datos invalidos devuelve 422
✓ actualizar director existente
✓ actualizar director inexistente devuelve 404
✓ eliminar director existente
✓ eliminar director con peliculas asociadas

Tests:      8 passed (13 assertions)
Duration:   0.19s

○ usuario@MV-Ubuntu:~/Descargas/peliculapp-main$ 

```

He tenido que crear en la base de datos un factory para simular

tests/Feature/Peliculas/


```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
• usuario@MV-Ubuntu:~/Descargas/peliculapp-main$ php artisan make:test Peliculas/PeliculaApiTest
  INFO Test [tests/Feature/Peliculas/PeliculaApiTest.php] created successfully.
○ usuario@MV-Ubuntu:~/Descargas/peliculapp-main$
```

```
• usuario@MV-Ubuntu:~/Descargas/peliculapp-main$ php artisan make:factory DirectorFactory
  INFO Factory [database/factories/DirectorFactory.php] created successfully.
○ usuario@MV-Ubuntu:~/Descargas/peliculapp-main$
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
usuario@MV-Ubuntu:~/Descargas/peliculapp-main$ php artisan test tests/Feature/Peliculas/PeliculaApiTest.php
✓ crear película asociada a director existente
✓ crear película con director inexistente devuelve 422
✓ actualizar película
✓ eliminar película
✓ mostrar película incluye datos del director

Tests: 6 passed (18 assertions)
Duration: 0.18s
○ usuario@MV-Ubuntu:~/Descargas/peliculapp-main$
```

tests/Feature/Security/

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
• usuario@MV-Ubuntu:~/Descargas/peliculapp-main$ php artisan make:test Security/SecurityApiTest
  INFO Test [tests/Feature/Security/SecurityApiTest.php] created successfully.
○ usuario@MV-Ubuntu:~/Descargas/peliculapp-main$
```

```

tests > Feature > Security > SecurityApiTest.php
11 class SecurityApiTest extends TestCase
12 {
13     use RefreshDatabase;
14
15     protected $user;
16     protected $token;
17
18     protected function setUp(): void
19     {
20         parent::setUp();
21         $this->user = User::factory()->create();
22         // Generamos el token JWT para el usuario
23         $this->token = auth('api')->login($this->user);
24     }
25
26     /** 23. Token expirado devuelve 401 */
27     public function test_token_expirado_devuelve_401()
28     {
29         // Forzamos la invalidación del token simulando que ha caducado
30         auth('api')->logout();
31         // Al hacer logout, el token actual queda en la "lista negra" (blacklist) de JWT,
32         // lo que emula exactamente el mismo comportamiento y respuesta que un token expirado.
33
34         $response = $this->withHeader('Authorization', "Bearer $this->token")
35             |>getJson('/api/directores');
36
37         $response->assertStatus(401);
38     }
39
40     /** 24. Respuestas de error no exponen stack trace en producción */
41     public function test_respuestas_de_error_no_exponen_stack_trace()
42     {
43         // Simulamos un entorno de producción seguro

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

● usuario@MV-Ubuntu:~/Descargas/peliculapp-main$ php artisan test tests/Feature/Security/SecurityApiTest.php

PASS Tests\Feature\Security\SecurityApiTest
✓ token expirado devuelve 401
✓ respuestas de error no exponen stack trace
✓ password no aparece en respuesta me

Tests: 3 passed (8 assertions)
Duration: 0.15s

○ usuario@MV-Ubuntu:~/Descargas/peliculapp-main$

```

Fase 2

CI/CD

```

1  name: Continuous Deployment
2
3  on:
4    push:
5      branches: [ "main" ]
6
7  permissions:
8    contents: read
9    packages: write
10
11 jobs:
12   deploy:
13     runs-on: ubuntu-latest
14     steps:
15       - name: Checkout Code
16         uses: actions/checkout@v4
17
18       - name: Log in to GitHub Container Registry
19         uses: docker/login-action@v3
20         with:
21           registry: ghcr.io
22           username: ${ github.actor }
23           password: ${ secrets.GITHUB_TOKEN }
24
25       - name: Downcase Repo Owner for GHCR
26         run: echo "REPO_OWNER=${GITHUB_REPOSITORY_OWNER,,}" >> $GITHUB_ENV
27
28       - name: Build and Push Docker Image
29         uses: docker/build-push-action@v5
30         with:
31           context: .
32           push: true
33           tags: |

```

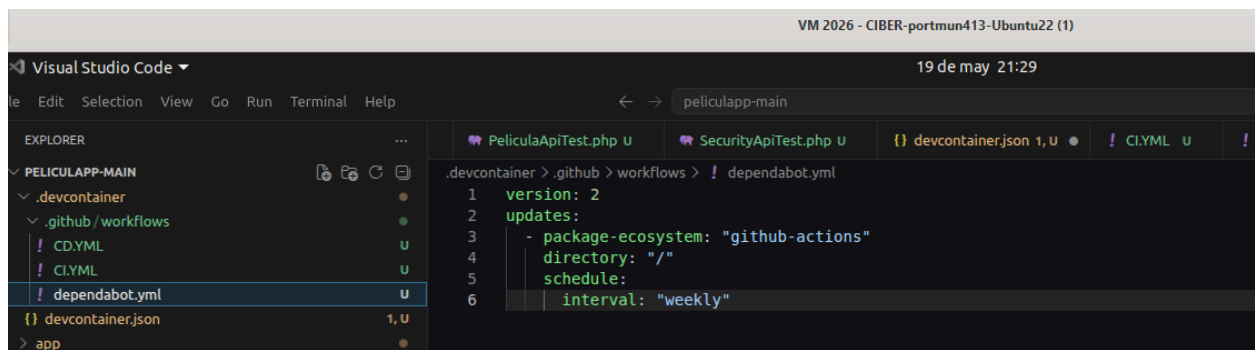
Configuramos los pasos automáticos que ocurren únicamente cuando subes algo definitivo a la rama main

```

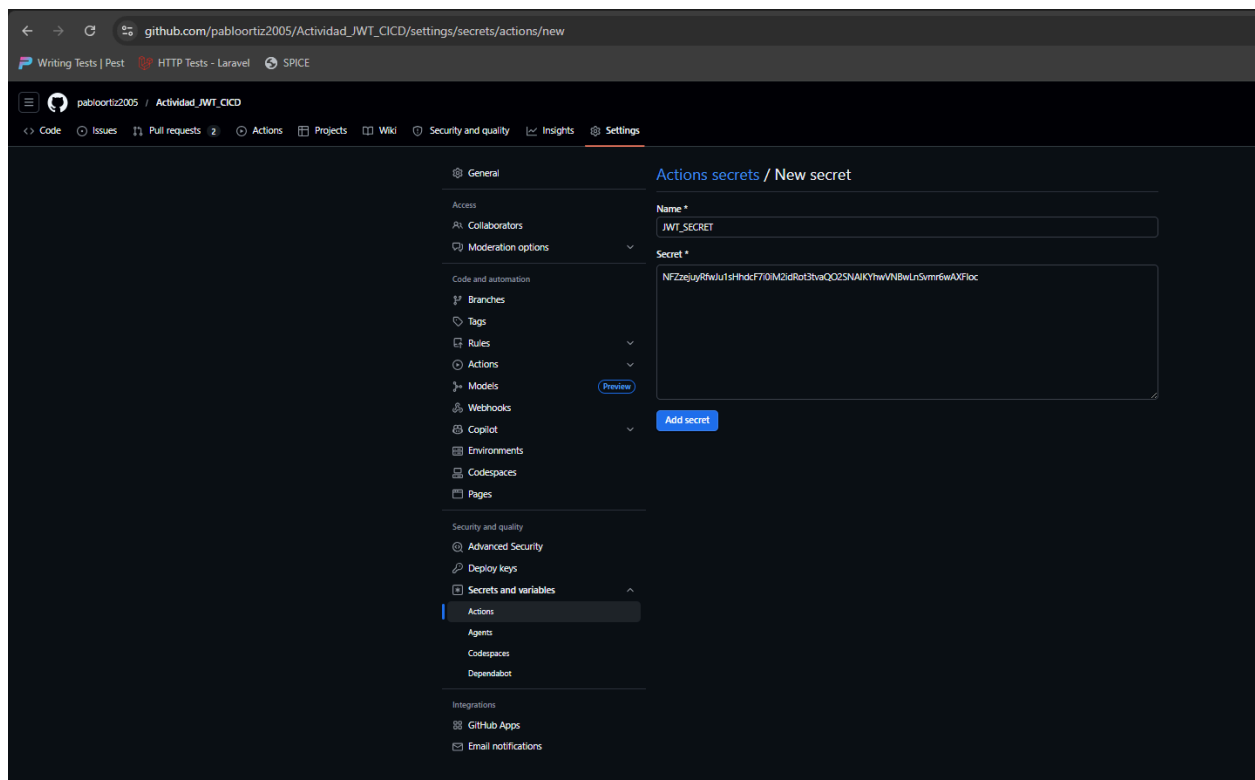
62   - name: Checkout Code
63     uses: actions/checkout@v4
64
65   - name: Setup PHP
66     uses: shivammathur/setup-php@v2
67     with:
68       php-version: '8.3'
69
70   - name: Audit Composer Dependencies
71     run: composer audit
72
73 build:
74   runs-on: ubuntu-latest
75   needs: [test, security]
76   steps:
77     - name: Checkout Code
78       uses: actions/checkout@v4
79
80     - name: Build Docker Image
81       run: docker build -t peliculapp:test .

```

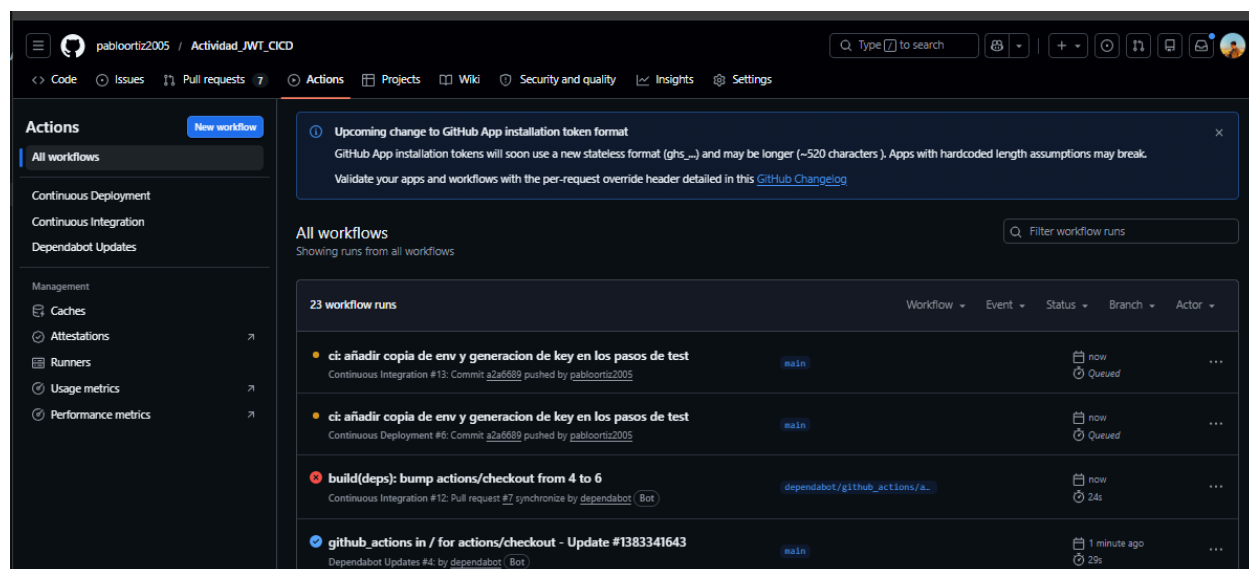
La Integración Continua es un sistema de auditoría automática. En lugar de que los desarrolladores ejecuten los tests en sus ordenadores antes de subir el código, GitHub Actions lo hace de manera obligatoria y en un entorno completamente limpio cada vez que haces git push o abres un Pull Request hacia las ramas main o develop



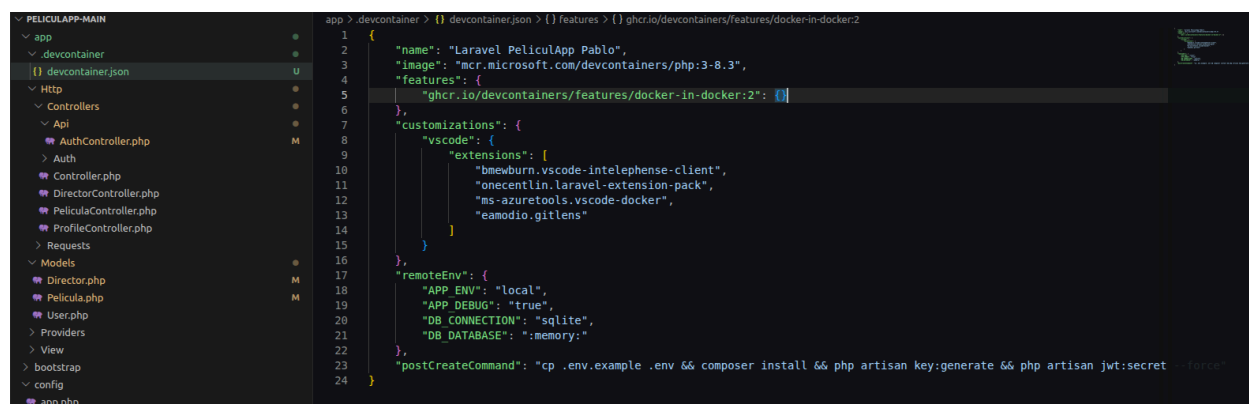
Dependabot es un bot oficial de GitHub que trabaja en segundo plano de forma automatizada para revisar el proyecto todas las semanas a través del archivo



CI busca asegurar la calidad del código, mientras que CD busca automatizar la entrega de ese código a los usuarios.



Dev Containers



Cuando abres el proyecto, este archivo le da órdenes a VS Code para hacer lo siguiente:

- Descarga la máquina ideal: Descarga una imagen oficial de Linux con PHP 8.3 ya instalado de fábrica.
- Instala tus herramientas visuales: Le inyecta a tu VS Code local las extensiones necesarias para Laravel de forma automática.
- Configura el entorno: Setea las variables para que la base de datos sea SQLite en memoria (:memory:), evitando que tengas que configurar un MySQL local.

- Ejecuta el postCreateCommand: Al arrancar, el contenedor ejecuta por sí solo:
- Copia el archivo .env.example a .env.
- Ejecuta composer install para descargar las librerías.
- Genera las claves de seguridad de Laravel (key:generate) y de los tokens (jwt:secret).

Funciona el dev container

